NUS Orbital 2026

Milestone 01

Poster Video App Logs

Atlas (Team ID:)

Wang Qichen (A0337219B)
Feng Yi (A0322277E)

Proposed Level of Achievement:
Artemis

Contents

Deployment 3

Motivation 3

Our Aim 3

User Stories 3

Overall Architecture Diagram 4

Core Features 5

Repository and Project Foundation 6

Project Structure and Version Control 6

Authentication and Protected Access 7

Email, Demo, and Google Sign-In 7

Protected Routes 7

Campus Map Rendering 9

MapLibre and OSM Tiles 9

Route Tracing 9

Mobile Frontend Experience 11

Responsive Layout 11

UML Diagram 12

Authentication Flow 12

Map Search and Routing Flow 12

Milestone 01 Progress 12

Planned Next Steps 13

Deployment

The deployed web application is available at: orbital-artemis-armap-nus.onrender.com

Motivation

Navigating through the NUS campus can be difficult because complete geospatial and indoor facility data is not always publicly available or intuitive to use. Even after reaching the correct building, students may still need to locate lecture rooms, tutorial classrooms, study spaces, restrooms, lifts, printing services, and other facilities.

These navigation needs are closely tied to students' daily routines and schedules. Atlas is motivated by the need for a smarter, more context-aware campus assistant that supports both outdoor movement across NUS and indoor discovery within selected buildings.

Our Aim

Atlas aims to create a seamless campus assistant for NUS students. The project combines map-based navigation, facility discovery, schedule-aware guidance, and a daily assistant interface so that students can move through campus more confidently.

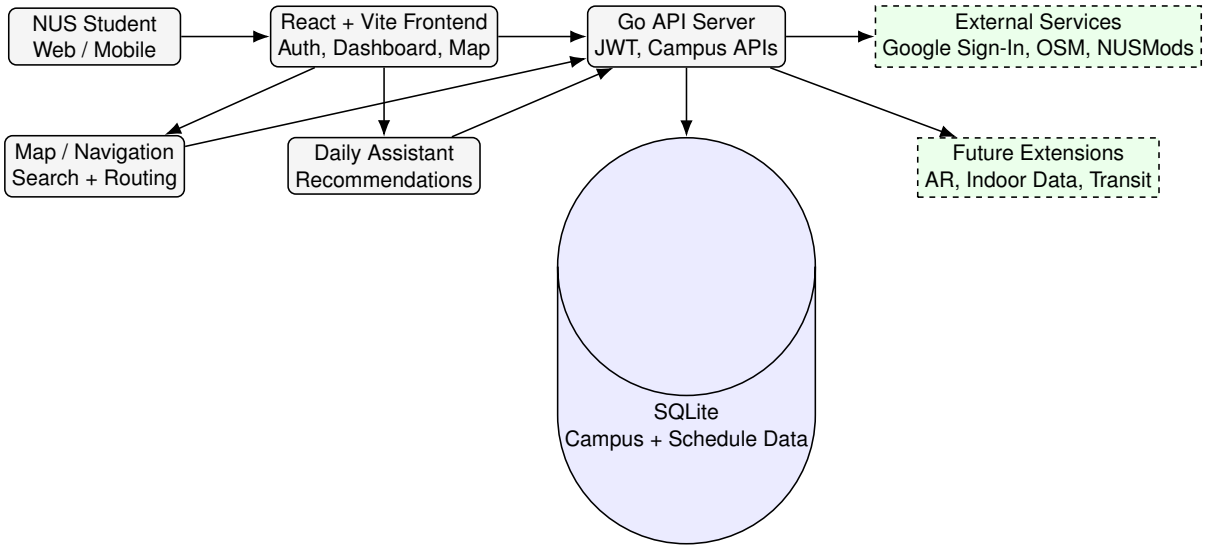
For Milestone 01, our goal is to establish a working full-stack prototype: authentication, a protected dashboard, campus data APIs, map rendering, location search, and deployment. Later milestones will extend this foundation toward richer AR-style navigation, indoor guidance, and smarter route planning.

User Stories

1. As an NUS student who wants to get to class on time, I want to use Atlas to navigate from my current location to the correct building and classroom so that I can reach lessons with less confusion.
2. As an NUS student who is unfamiliar with a campus building, I want to find indoor facilities such as restrooms, lifts, study spaces, and printing services so that I can quickly locate useful services.
3. As an NUS student managing a busy day, I want schedule-aware reminders and location-based suggestions so that I know where to go next and when to leave.

Overall Architecture Diagram

The architecture follows the proposal's modular design: a user-facing interface, an authentication layer, a campus assistant/dashboard layer, a navigation and map layer, backend APIs, persistent data, and external campus services.



Atlas overall architecture for Milestone 01 and planned Artemis extensions

Core Features

The feature list below is based on the GitHub issue tracker. Sprint 3 issues are intentionally excluded from the Milestone 01 core feature count, including OpenTripPlanner backend setup, NUS bus API setup, multimodal route integration, and route color coding.

S/N	Feature	Completed At	Brief Explanation
Core Features			
1	Repository and project foundation	Milestone 01	Issue #1 established the repository, README skeleton, license, and basic development structure.
2	Authentication and protected access	Milestone 01	Issues #2 and #8 cover email/password login, Google Sign-In, JWT issuing, and protected dashboard routing.
3	Campus map rendering	Milestone 01	Issues #7 and #17 cover OpenStreetMap/MapLibre rendering and fixing building polygon display.
4	Location search and route tracing	Milestone 01	Issue #12 adds a searchable map interface with suggestions, map centering, and route-oriented interaction.
5	Daily assistant and campus data UI	Milestone 01	Issue #11 connects schedule, recommendations, buildings, facilities, and sync status into a student-facing assistant dashboard.
Supporting / Usability Features			
6	Mobile frontend experience	Milestone 01	Issue #10 tracks responsive layout and mobile usability checks for auth, dashboard, map, schedule, and recommendations.

Repository and Project Foundation

This feature establishes the engineering foundation for Atlas. From the user's point of view, it makes the project accessible as a real deployed application rather than a static proposal: the repository contains reproducible setup instructions, a clear project structure, and a production deployment path.

Project Structure and Version Control

Atlas is organized into a frontend, backend, deployment configuration, and supporting documentation. This mirrors the modular design described in the proposal and keeps responsibility boundaries clear. The frontend owns user interaction, the backend owns authentication and campus data APIs, and the deployment files define how the system runs in production.

```
Orbital_Artemis_ArMap_Nus/
|-- frontend/ React + Vite frontend and mobile app shell
| |-- src/ Auth, dashboard, campus map, and UI code
| `-- ios/ Capacitor iOS project
|-- backend/ Go API server
| |-- cmd/server/ Server entry point
| `-- internal/atlas/ Auth, campus data, routing, and API logic
|-- deploy/ Production deployment configuration
|-- poster_work/ Poster and presentation assets
|-- Dockerfile Container build definition
|-- docker-compose.yml Local multi-service run configuration
`-- readme.tex Milestone report source
```

Project Structure

The repository uses GitHub issues and pull requests as progress logs. Issue #1 initialized the repository and documentation baseline, while later issues track authentication, map rendering, search, dashboard integration, and mobile usability. This supports the proposal's branch-based workflow: tasks are decomposed into issues, implemented on feature branches, reviewed through pull requests, and eventually integrated into `main`.

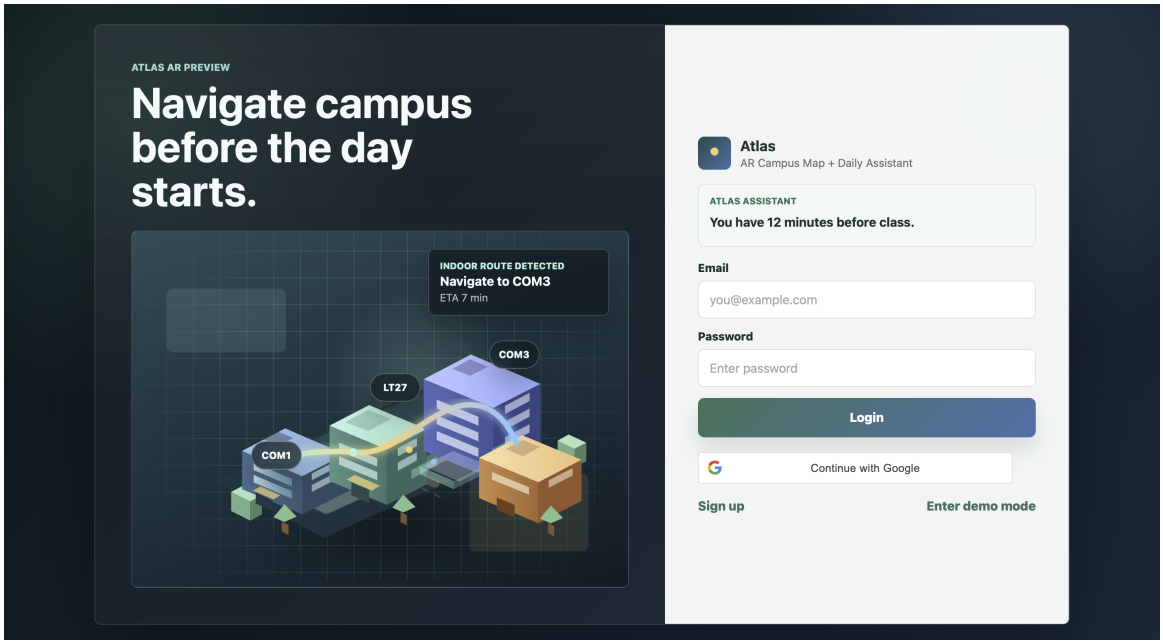
The main trade-off of this structure is that it creates more files and coordination overhead than a single-folder prototype. However, the benefit is significant for an Artemis-level project: each module can evolve independently, deployment can be reproduced, and later AR/navigation extensions can be added without rewriting the whole codebase.

Authentication and Protected Access

Authentication allows students to enter Atlas through email/password login, registration, demo mode, or Google Sign-In. Once authenticated, users can access protected pages such as the dashboard and map.

Email, Demo, and Google Sign-In

The login system supports multiple entry paths. Email/password login provides a traditional account flow, demo mode allows quick testing without setup, and Google Sign-In supports a more realistic student login experience.



Authentication Screen

On the backend, Atlas verifies credentials and issues a JWT. For Google Sign-In, the frontend obtains a Google ID token and sends it to the backend, where it is verified against the configured Google client ID. If verification succeeds, the backend returns the same application JWT format used by email login. This keeps the rest of the application independent of the authentication method.

From a software engineering perspective, this follows separation of concerns. The frontend handles user interaction and token storage, while the backend performs trust-sensitive verification and token signing. This is safer than trusting the frontend alone, because the backend remains the authority for protected access. The trade-off is that local development requires OAuth configuration and network access to Google's verification service, but the design is closer to a production-ready authentication system.

Protected Routes

Protected routes guard the dashboard and map pages. If a user is not authenticated, the frontend redirects them to the login screen. API requests also attach the JWT in the `Authorization` header, allowing the backend to reject unauthenticated access consistently.

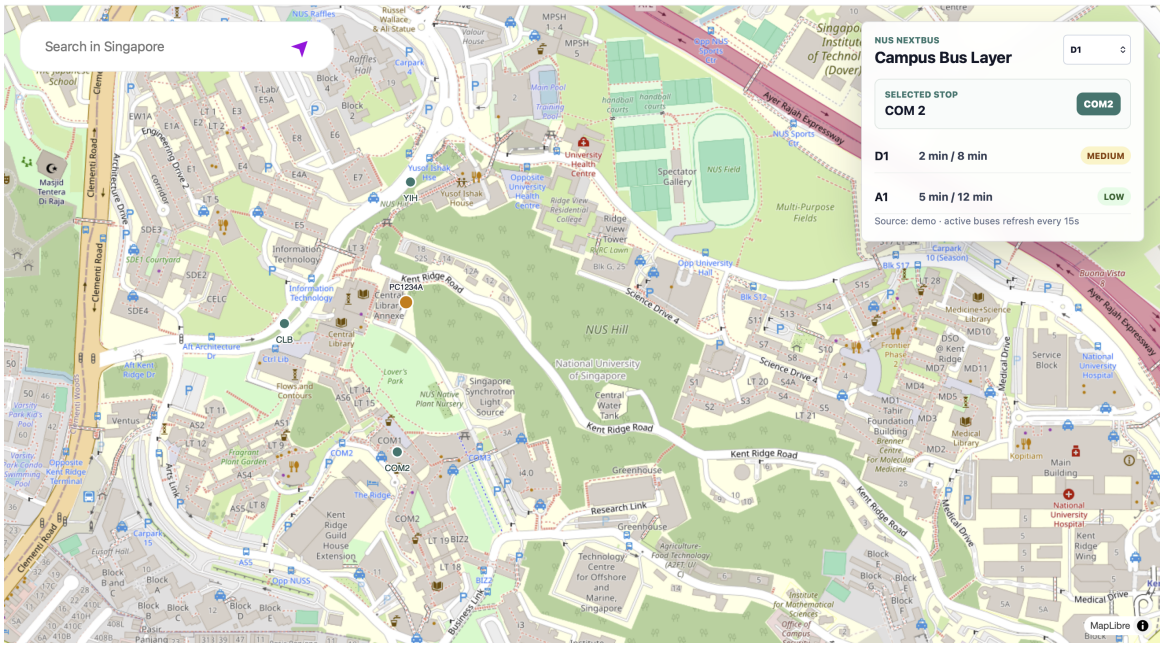
This pattern reduces duplicated access checks. Instead of manually checking login state inside every page component, protected routing creates a reusable access boundary. It also makes later feature additions easier, because new protected pages can reuse the same wrapper.

Campus Map Rendering

The map feature gives users an interactive visual interface for campus navigation. For Milestone 01, Atlas renders a MapLibre/OpenStreetMap-based campus map centered around NUS and prepares the map surface for future AR and indoor navigation work.

MapLibre and OSM Tiles

The frontend uses MapLibre to render map tiles and manage markers, routes, and future geospatial overlays. This gives the project an open-source map engine that can be extended with custom campus data instead of relying only on a closed map UI.



Campus Map Screen

The map logic is separated into a map initialization module, a `MapEngine` helper, routing services, and UI components. This modular approach prevents the main map page from becoming a single large component. It also makes it easier to add bus layers, indoor facility overlays, or AR-style markers later.

The bug tracked in issue #17 showed why this separation matters. Rendering geospatial polygons and map layers can fail due to data format mismatches, layer ordering, or source initialization timing. By separating map engine actions from UI state, Atlas can debug and update rendering logic without changing authentication or dashboard code.

Route Tracing

Route tracing allows selected start and end points to be drawn on the map. In Milestone 01, this is implemented as a route line on the MapLibre layer. It is an early prototype of the proposal's broader navigation goal.

The engineering trade-off is that route tracing is currently web-map based rather than full AR. This is intentional for Milestone 01: a reliable map and routing foundation reduces risk before

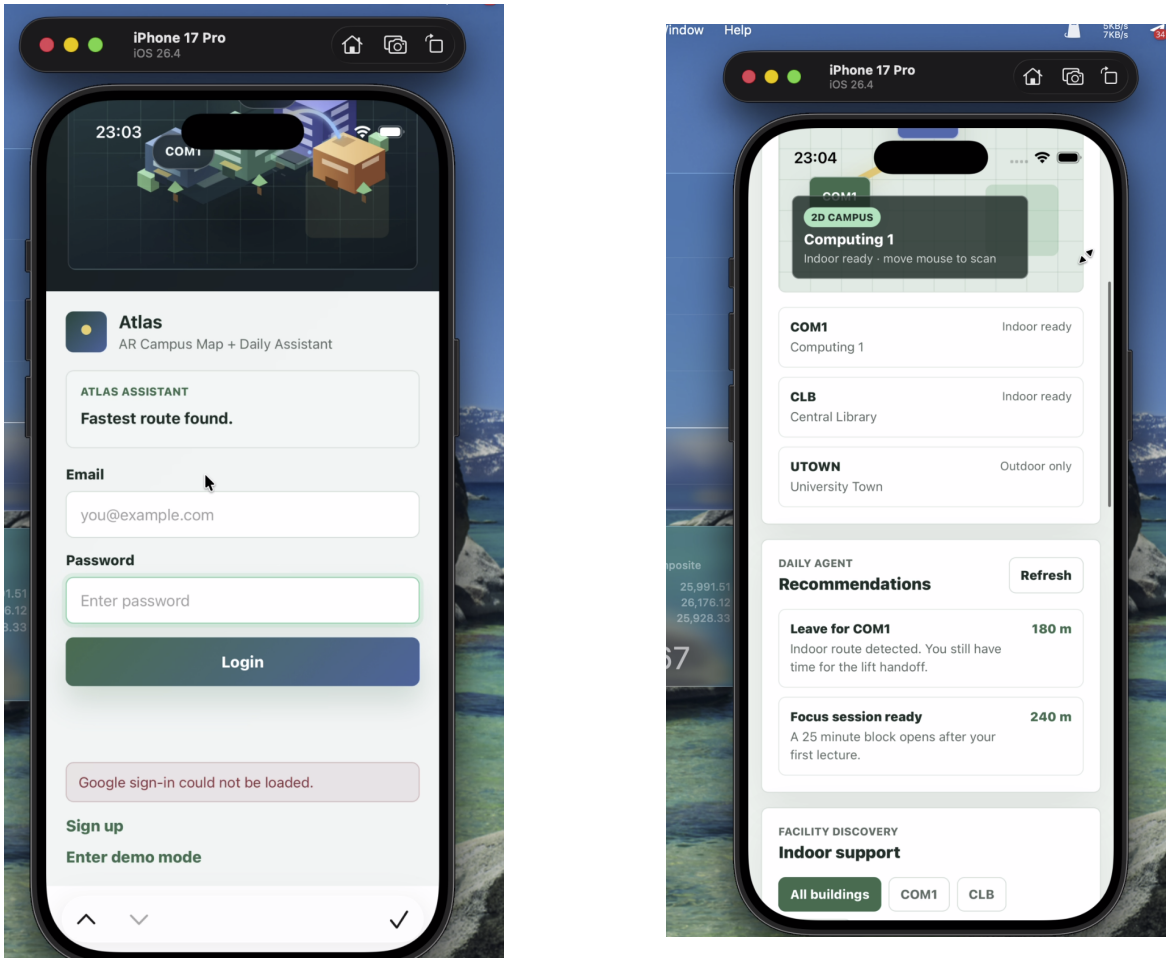
adding heavier AR components in later milestones.

Mobile Frontend Experience

Because campus navigation is often used while students are moving, mobile usability is a supporting but important feature. Issue #10 tracks responsive layout and manual viewport checks for authentication, dashboard, map, schedule, and recommendations.

Responsive Layout

The frontend is designed to remain readable on common mobile widths. Cards, buttons, filters, and map panels are arranged to avoid horizontal scrolling and text overlap.



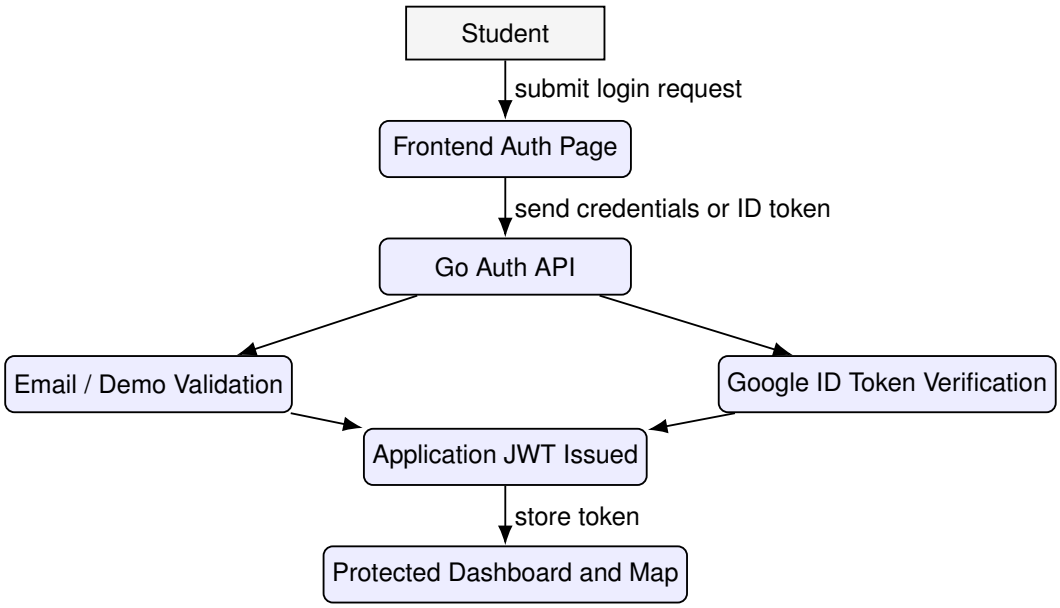
Mobile App Screens

The main engineering principle here is progressive enhancement: the application should remain usable on desktop while also adapting to mobile. The trade-off is extra CSS and layout testing, but this matters for a campus assistant because students are more likely to use it on phones than on large desktop screens.

UML Diagram

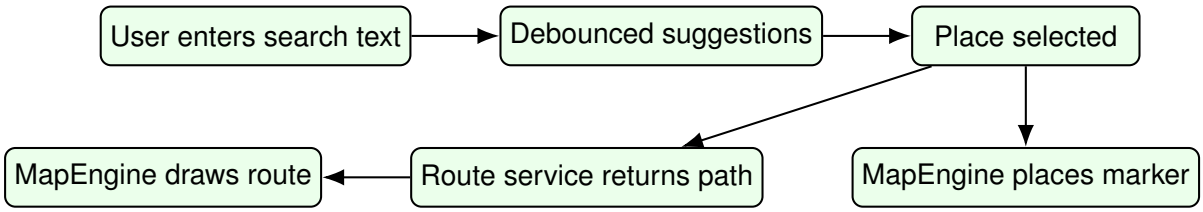
The diagrams below summarize two important interactions in the current prototype: authentication and map search/routing.

Authentication Flow



Authentication sequence for protected access

Map Search and Routing Flow



Map search and route drawing workflow

Milestone 01 Progress

Milestone 01 focused on ideation, planning, initial prototyping, and establishing a deployed technical foundation. The current implementation provides a working React frontend, Go backend, SQLite data layer, authentication flow, dashboard, map page, campus data APIs, and Render deployment.

The implementation also validates the proposed modular system design. Frontend pages, backend handlers, authentication logic, campus data storage, and map features are separated into maintainable modules, making it easier to extend the project in later milestones.

Planned Next Steps

- Expand the navigation layer toward richer indoor guidance and AR-style interaction.
- Improve campus data coverage for buildings, rooms, facilities, and navigation points.
- Continue the Sprint 3 work on transit, multimodal routing, and route visualization as extension features.
- Gather user feedback from NUS students and refine the assistant workflow.